

QA Engineering Take-Home Assignment

Part 1 — CRM Workflow Analysis

Task 1.1 — Lead-to-Deal Lifecycle

End-to-end flow

Zoho CRM: a Lead is captured (web form, manual entry, or import) in the Leads module. Working the lead — calls, emails, status updates — happens entirely on the Lead record. When qualified, the rep clicks Convert. This opens a dedicated conversion screen with three toggles: create a new Contact, create/link an Account, and optionally create a Deal in the same step. The rep can edit field mappings on this screen before confirming.

HubSpot CRM: HubSpot's core object is the Contact, not a separate Lead record. A new contact typically enters at a low lifecycle-stage value (e.g. Subscriber/Lead) via a form fill, import, or manual add. Instead of a conversion step, the rep manually advances the Lifecycle Stage property (or a workflow does it based on behaviour) and creates a Deal from the Contact or Company record, associating it explicitly. If HubSpot's separate "Leads" object is enabled on the portal, a similar convert-to-deal action exists, but the classic and still most common pattern is contact + lifecycle stage + manually created Deal.

Auto-populated vs. manually entered

Step	Auto-populated	Manual entry
Zoho conversion	Contact name, email, phone, company/account name, lead source copied per field-mapping rules	Deal name (not derived automatically), deal stage, deal amount, closing date
HubSpot deal creation	Associated company/contact populated from the record you launched the deal from	Deal name, pipeline, stage, amount, close date — none are inferred from the contact

Behavioural differences (Zoho vs. HubSpot)

- Duplicate detection on conversion: Zoho actively checks the Lead's email/company against existing Contacts/Accounts at the moment of conversion and offers to merge or link to the existing record. HubSpot has no equivalent conversion-time check because there is no separate Lead-to-Contact step — deduplication instead happens continuously in the background via its dedupe/merge suggestions tool, so a near-duplicate contact can persist for a while before HubSpot ever flags it.
- Explicit vs. implicit record creation: Zoho's conversion is a single transactional action that stamps out Contact/Account/Deal together and is logged as a conversion event on the Lead. HubSpot's path is several independent actions (edit lifecycle stage, then separately create/associate a Deal), so there is no single audit event that says "this lead became this deal."
- Deal creation is optional vs. deliberate: In Zoho, a rep can convert a Lead without creating a Deal, which is easy to do accidentally by leaving that toggle off — the pipeline never gets an entry even though the rep believes the lead was "converted." In HubSpot, a Deal never appears unless someone deliberately creates one, so there's no accidental-skip risk, but it means lifecycle stage and pipeline stage can drift out of sync (a contact marked "Customer" with no closed-won Deal behind it).

Where data can be silently lost or overwritten

- Zoho: if the same field is mapped from Lead to both Contact and Account (e.g. Phone), and the Account already exists, the existing Account's phone value is overwritten by the incoming lead's value with no diff or confirmation shown.
- Zoho: Lead Source and Lead Owner history live only on the Lead record; once converted, that context is not automatically carried onto the Deal unless the mapping explicitly includes it — teams often lose "how this deal originated" at the pipeline-reporting level.
- HubSpot: lifecycle stage is a single-value property with no automatic "only move forward" rule by default. A later form submission, list import, or workflow can silently regress a contact from Customer back to Lead if the automation isn't explicitly guarded, which quietly corrupts funnel reporting.

Task 1.2 — Contact & Account Management Workflow

Creating a Contact under an existing vs. new Account

Zoho: the Contact record has an Account Name lookup field. Typing an existing Account name shows a searchable picker; typing a name with no match lets the rep create a brand-new Account inline. Validation is limited to required-field checks — Zoho does not warn if the new Account name is a near-duplicate of an existing one (e.g. "Acme Inc" vs "Acme, Inc.").

HubSpot: Contacts associate to Companies via the Associations panel; HubSpot suggests a likely Company automatically based on the contact's email domain, which is a useful soft validation most CRMs don't offer. A rep can still associate to the wrong company or skip association entirely with no hard block.

Same email used for two different contacts

Platform	Behaviour
Zoho CRM	Allows duplicate emails by default. Duplicate handling is opt-in: an admin must configure the Email field as a "unique" field or set up Duplicate Check rules for CRM to block or warn on save. Out of the box, two Contacts can silently share an email.
HubSpot CRM	Email is the primary de-duplication key for Contacts by default. Attempting to create a second contact with an email already in the system does not create a duplicate — HubSpot routes the update into the existing record instead. This is safer for data integrity but can surprise a rep who intended to create a distinct new person (e.g. a shared team inbox address).

Deleting or merging an Account

- Zoho: deleting an Account does not automatically delete related Contacts and Deals — they remain but lose the Account link, effectively becoming orphaned unless the admin has configured cascading delete. Merging two Accounts moves related Contacts/Deals/Activities to the surviving record, but any field-level conflicts are resolved by a manual pick during the merge screen, with the losing record's non-picked values discarded permanently.
- HubSpot: deleting a Company does not delete associated Contacts or Deals; they simply lose the company association and remain fully intact and searchable. Merging two Companies keeps the primary record's property values by default unless the user manually overrides per property, and all associations from the merged-away Company roll over to the surviving one.

"Primary contact" for an Account

Zoho does not have a built-in "Primary Contact" designation on the Account layout out of the box — teams typically add a custom lookup field or rely on the related-list order, which is easy to leave inconsistent across

Accounts. HubSpot has a native primary-company association on the Contact and, going the other direction, association labels (e.g. "Primary Contact") can be set explicitly between a Company/Deal and a Contact, so the concept is a first-class, queryable property rather than a workaround.

Task 1.3 — Email & Activity Logging Workflow

Where a logged email appears

Sending an email from within a HubSpot Contact record logs it to that Contact's timeline. If the Contact is already associated with an open Deal, HubSpot also surfaces the email on the Deal's timeline automatically, because the timeline pulls activity from every object the Contact is associated with at send time. If the association is added after the email was sent, the email does not retroactively appear on the Deal — it stays tied to the Contact only.

Zoho behaves similarly for emails sent from the Contact page, but cross-posting to the related Deal is less automatic: unless the email is sent from the Deal's own "Emails" related-list tab (or the Deal is explicitly the context the send happened in), it may only show under the Contact, requiring a rep to check both records to get the full picture.

Orphaned activities

Logging a call or note without associating it to a Deal does not make it invisible in either CRM — it still lives on the Contact/Company timeline. True orphaning happens one level down: an activity created via API or bulk import with no Contact/Company/Deal association at all becomes unreachable from any standard record page and only surfaces through a direct search or report — effectively invisible to a rep browsing normally.

BCC-based email capture

- Both platforms provide a personal BCC address (e.g. HubSpot's `*@bcc.hubspot.com`, Zoho's mail-forwarding/BCC address) that logs the email against whichever contact record matches the recipient's email address.
- Risk: matching is done purely on email address string. If a rep BCCs a thread that includes a personal Gmail alias or a shared distribution address not yet in the CRM, HubSpot will auto-create a new contact for that address (adding CRM clutter), while Zoho may simply fail to log the email against any record at all if strict matching finds no contact — a silent drop rather than an error.
- Risk: if two different Contacts in the CRM were ever entered with the same email address (see Task 1.2 — possible in Zoho), a BCC-captured email can attach to the wrong one of the two, and there is no UI indicator that the match was ambiguous.

Activity timeline UX — what a QA engineer could miss relying on the UI alone

- Both CRMs collapse older activity behind a "load more" / pagination action, so a QA engineer scanning only the visible timeline can miss activity that occurred outside the default window.
- Failed sends (bounced email, blocked automation email) are often represented with the same visual weight as successful ones, or surfaced only via a small icon/tooltip — easy to misread as "delivered" during a quick UI pass.
- Activities logged against a record via API/integration (marketing tools, dialers) can appear with generic or missing metadata (no owner, no timestamp precision), making it hard to visually confirm they are complete or correctly attributed without opening the record detail.

Task 1.4 — Automation & Workflow Rules

Automation built for this task: "When a Deal moves to the Negotiation stage, send an internal email notification to the deal owner and create a follow-up task."

Setup steps and available building blocks

	Zoho CRM (Workflow Rules)	HubSpot CRM (Workflows)
Trigger	Module = Deals, "On a record action" → field update, condition: Stage = Negotiation	Enrollment trigger: Deal property Stage is any of Negotiation
Conditions	Field-based criteria builder; can add multiple AND/OR blocks	Filter branches, plus enrollment criteria (property-based, event-based)
Actions available	Email alert (template-based), task, field update, webhook, custom function	Send internal notification, create task, update property, send automated email, webhook, branching logic
Where it's built	Setup → Automation → Workflow Rules	Automation → Workflows

Intentionally breaking it

- Delete the associated Contact: in Zoho, the Deal record survives and the workflow still fires on stage change, but any email personalization token referencing the Contact (e.g. {{Contact.First_Name}}) renders blank — the email still sends but looks broken, with no error surfaced to the rep. In HubSpot, if the notification/email step references a Contact-level property and the association is gone, the action either sends with empty personalization tokens or is skipped, and this is visible only in the workflow's own execution history — not anywhere a standard user would look.
- Remove/rename the deal stage used in the trigger condition: in both platforms the workflow's condition silently stops matching — the automation is not disabled, it simply never fires again, and there is no proactive alert to whoever owns the process. This is one of the most dangerous failure modes because everything looks "active" in the automation list.
- Disable the email template: Zoho shows the workflow rule as still Active, but the email action fails at execution time; the failure is only visible in the rule's execution log (Setup → Automation → Workflow Rules → Rule Log), which is not exposed to standard sales users. HubSpot's Workflow History tab shows a clear per-enrollment error state for a failed action, but that tab is gated behind marketing/automation permissions most reps don't have.

Does it fail silently, and where would QA look?

In both CRMs the answer is: yes, it can fail silently from an end-user's perspective. The only reliable way to confirm a workflow actually ran (versus not enrolling, versus enrolling and erroring) is the automation's own execution/history log, which is admin- or permission-gated in both tools. A QA engineer validating this feature must check the automation log directly rather than inferring success from the Deal record alone.

Gaps in the audit trail

- Neither CRM surfaces automation failures on the record itself (e.g. a banner on the Deal saying "a workflow action failed") — the failure only exists in a separate automation log a typical user never opens.
- Zoho's workflow rule log retention and level of detail (e.g. exact reason an email action failed) is limited compared to HubSpot's per-enrollment history, making root-cause debugging slower in Zoho for anything beyond "it didn't fire."

Task 1.5 — Reporting & Dashboard Workflow.

Sync delay for a newly added deal

A Deal created directly in Zoho CRM's native reporting appears in a standard report on next refresh, effectively real time — native Zoho reports query live CRM data. Where a delay does appear is if the org also pushes data into Zoho Analytics (a separate BI layer often used for more complex dashboards); that sync runs on a schedule (commonly hourly or nightly depending on plan), so a deal can be live in the CRM but absent from an Analytics-based dashboard for a noticeable window — a distinction that is easy to miss during testing if you don't know which reporting layer you're looking at.

Editing close date or amount

Changes to close date or amount are reflected immediately in native, on-the-fly Zoho reports on next load. Reports built on a saved dataset/snapshot (again, typically an Analytics-layer report) will not reflect the change until the next scheduled sync, which can mislead a sales manager checking numbers mid-day into believing the pipeline hasn't moved.

Custom property filtering (HubSpot cross-check)

In HubSpot, creating a new custom deal property and filtering a report on it works correctly for deals created after the property existed. Deals created before the property was added simply have a null value for it, and a report filter set to "is equal to X" silently excludes those legacy deals rather than flagging them as "not set" — a manager could read the filtered count as the true total and be materially wrong for older pipeline.

A scenario where a report could mislead a sales manager

An "Expected Revenue This Month" report that simply sums Amount $\times$ default stage probability can overstate the real forecast: stage-based default probabilities are static assumptions (e.g. every deal in "Negotiation" counted at 75%) regardless of how long the deal has actually sat there or whether it's stalled. A deal that has been stuck in Negotiation for four months is weighted the same as one that entered yesterday, inflating the forecast with deals that are functionally at risk. The correct fix is either a probability model that decays with time-in-stage / recalculates from deal-level signals, or a report view that separately flags stale deals (no activity for N days) so the manager can discount them manually rather than trusting a single blended number.

Task 1.6 — Spot the Logical Bugs

Four data-integrity/logical issues surfaced across Tasks 1.1–1.5, reported in the format requested.

Bug 1 — Deals can exist with no Owner, and they are silently excluded from rep-level pipeline reports

Steps to reproduce: 1) Create a Deal via bulk import without mapping the Owner column, or remove the Owner from an existing Deal where permissions allow a blank value. 2) Open a standard "Pipeline by Owner" or "Deals by Rep" report.

Actual behaviour: The unowned Deal either drops out of every owner-filtered report entirely, or is bucketed into a generic "Unassigned" group depending on report type, with no alert raised anywhere in the CRM that an unowned Deal exists.

Expected behaviour: The system should either block saving a Deal without an Owner, or surface unowned Deals prominently (e.g. a standing view/alert) so they can't quietly disappear from pipeline accountability.

Business impact: Real pipeline value goes untracked and unworked — nobody is accountable for progressing the deal, and management-level revenue forecasts undercount true open pipeline.

Bug 2 — A Deal can be marked Closed Won with required-adjacent fields left empty when saved via import or API

Steps to reproduce: 1) Attempt to set a Deal's Stage to Closed Won through the normal UI with Amount blank — field validation typically blocks this. 2) Perform the same update via bulk import or API instead.

Actual behaviour: Field-level "required for this stage" validation is enforced in the interactive UI layer but not consistently re-applied on import/API writes, allowing a Closed Won deal to be saved with Amount (or other stage-required fields) empty.

Expected behaviour: Business-rule validation tied to a stage (e.g. "Amount required to close Won") should be enforced server-side for every write path, not only the UI form.

Business impact: Revenue and win-rate reports become inaccurate — a \$0 or null-amount "Won" deal either undercounts closed revenue or, if excluded from aggregation entirely, overstates win rate by hiding an incomplete record among genuine wins.

Bug 3 — Reopening a Closed Lost Deal does not reset or clearly version its original Close Date, distorting sales-cycle-length reporting

Steps to reproduce: 1) Mark a Deal Closed Lost with a Close Date of, say, day 10. 2) Reopen the Deal and move it back into an active pipeline stage weeks later. 3) Eventually close it Won and check a "Sales Cycle Length" or "Days to Close" report.

Actual behaviour: The original Close Date is left in place (or silently overwritten by the reopen action, depending on configuration) with no explicit history of the reopen event on the standard record view, so cycle-length calculations either use a stale date or lose the fact that the deal was ever lost and revived.

Expected behaviour: Reopening a closed deal should be an explicitly logged event (timestamped), and cycle-time reporting should be able to account for the lost→reopened→won path rather than treating it as one continuous, uninterrupted cycle.

Business impact: Sales-cycle-length and win-rate trend reports are skewed — reopened deals make average cycle time look artificially short or long depending on which date survives, undermining forecasting models that rely on that metric.

Bug 4 — Automation execution logs are restricted to admin/automation-permission users, not visible to standard reps

Steps to reproduce: 1) As a standard (non-admin) sales user, have an automation rule fire on one of your own Deals. 2) Attempt to confirm, from your own login, whether the automation actually ran or failed.

Actual behaviour: There is no accessible view for a standard user to check whether an automation tied to their own record executed successfully; that information lives only in an admin-gated execution/history log.

Expected behaviour: Reps should be able to see at least a pass/fail status for automations that acted on their own records, even if full configuration/editing stays admin-only.

Business impact: Failed automations (missed follow-up tasks, unsent notifications) go undetected by the person closest to the deal, increasing time-to-detect and creating a support/ops bottleneck since only admins can diagnose "why didn't this task get created."

Part 2 — Deep Product Understanding

Task 2.1 — Mental Model of the Product

CRM chosen: Zoho CRM. Explained as if onboarding a new sales rep.

Think of Zoho CRM as a funnel with four connected record types. A Lead is a raw, unqualified spark of interest — someone who filled a form, was scraped from a list, or handed you a business card. It isn't yet proven to be a real, working opportunity, so it lives in its own module, separate from your "real" pipeline. Once you've had a conversation and confirmed the person and company are worth pursuing, you Convert the Lead — this is the one moment where Zoho takes that raw record and turns it into the CRM's real currency: a Contact (the person), an Account (the company they work for), and, if you're ready, a Deal (the actual sales opportunity you're now tracking toward Closed Won or Closed Lost).

The Lead vs. Contact distinction matters because it keeps your pipeline honest: only things that survived qualification show up as Contacts/Deals, so pipeline reports aren't polluted with cold, unqualified noise. The conversion moment is also the only place field data reliably crosses over — after that point, the Lead record is essentially archived, and everything happens on the Contact, Account, and Deal instead.

Where the CRM sits in the sales process: everything before the Lead exists (ads, campaigns, a website visitor who hasn't filled a form) is outside the CRM, usually in a marketing tool that feeds Leads in. Everything after Closed Won — invoicing, onboarding, support — technically continues in the Deal/Account/Contact records but is often handed off to other systems (billing, helpdesk) that sync back to the same Account. The CRM's core job is the middle: turning "someone showed interest" into "we have a documented, ownable, reportable sales process" that ends in a decision.

Task 2.2 — Feature Impact Assessment

Feature: "Add a mandatory manager approval before a Deal can be marked Closed Won."

Directly affected modules, automations, and reports

- Deals module — a new approval sub-process/status gates the Stage field transition to Closed Won; likely needs a new "Approval Status" property (Pending / Approved / Rejected).
- Any existing Workflow Rule or Blueprint transition that currently fires "on Deal Stage = Closed Won" — these need to be re-pointed to fire on Approval Status = Approved instead, or they will fire the moment the stage field changes, before approval is final.
- Forecasting and quota/commission reports that count revenue at the point Stage flips to Closed Won — these must be re-defined to only count post-approval, or reps will appear to have hit numbers that aren't actually confirmed.
- Email notification templates tied to "deal won" congratulations/handoff messages — same timing problem, they'd fire prematurely if left wired to the raw stage field.

Workflows that could break silently

- Any downstream automation that assumes "Stage = Closed Won" is a final, trustworthy state (e.g. auto-creating an onboarding task, notifying finance to invoice) will now fire on a deal that could still be rejected — producing real-world actions (invoices, kickoff emails) for revenue that isn't actually confirmed.
- Integrations that read the Stage field via webhook/API (billing systems, BI tools) will not automatically know about the new Approval Status field unless every integration is explicitly updated — they'll treat the deal as Won the moment the field changes.

Downstream data effects

- Forecasting: revenue-recognition timing shifts later (at approval, not at stage change), which can temporarily depress "deals won this period" numbers right after rollout if forecasts aren't re-baselined.
- Quota/commission dashboards: reps may see their number "stuck" in a pending state, which is a UX and trust risk if not clearly communicated — expect a spike in "why isn't my deal counted" tickets.
- Integrations: any system currently treating stage-change as the single source of truth for "deal is won" needs to be updated to consume the new approval status, or it will be wrong from day one.

Edge cases in the approval flow

- Approver is on leave: without an explicit delegation/escalation rule, every deal routed to them stalls indefinitely — needs an auto-escalation after N days or a designated backup approver.
- Approver is also the Deal owner: without a hard rule preventing self-approval, this creates a control gap that defeats the purpose of the feature (a rep could approve their own deal).
- Bulk deal updates via import: import processes often write directly to the Stage field and bypass UI-level workflow gating — if the approval rule isn't enforced at the database/API layer too, bulk import becomes a silent bypass of the entire control.

Priority test areas (6+)

#	Test area	Why it's a priority
1	Approval gate blocks Closed Won until approved	Core function of the feature — must be unbreakable via UI
2	Self-approval prevention	Direct control-integrity risk if the owner and approver are the same person
3	Bulk import / API bypass	Most likely silent hole — approval control not enforced outside the UI
4	Approver unavailable / delegation & escalation	Business continuity — deals must not stall indefinitely
5	Existing automations re-pointed correctly (no premature firing)	Prevents invoices/onboarding tasks firing on unapproved deals
6	Forecast & quota reports reflect approval, not raw stage	Prevents reps/managers trusting numbers that aren't final
7	Integration/webhook payload includes approval status	External systems must not treat stage-change as final truth
8	Notification timing (approver alert, reminder, rep notified of outcome)	Feature is unusable if approvers aren't reliably notified

Task 2.3 — Cross-Feature Dependency Reasoning

If only three features/workflows could be tested before a major release, these are the three I'd choose, in priority order.

1. Lead-to-Deal conversion & duplicate handling

Why high risk: Every downstream object — Contact, Account, Deal, every report built on them — depends on this step being correct. A bug here (silent overwrite, failed conversion, missed duplicate) corrupts data at the root of the funnel, and the damage compounds the longer it goes unnoticed.

Testing approach: Functional testing of the conversion screen plus targeted data-integrity testing: convert leads against pre-seeded duplicate Contacts/Accounts and assert exact field-level outcomes, not just "conversion succeeded."

Automation candidate: Yes, high ROI — this is a deterministic, high-frequency, high-blast-radius flow with clear expected outputs, making it ideal for an automated regression suite that runs on every release.

2. Automation/workflow rule execution & failure handling

Why high risk: Automations silently touch revenue-critical actions (notifications, tasks, field updates) with almost no visible audit trail to standard users, as shown in Task 1.4 — a broken automation can go undetected for weeks.

Testing approach: Integration testing that deliberately breaks pre-conditions (deleted association, disabled template, removed picklist value) and asserts on the execution log, not just the absence of a visible error.

Automation candidate: Partially — the trigger/action logic is automatable, but confirming the human-visible failure experience (or lack of it) benefits from periodic manual review since it's about UX/observability, not just pass/fail output.

3. Reporting/dashboard revenue and forecast accuracy

Why high risk: These numbers go straight to leadership decision-making; the failure mode isn't a crash, it's a believable-but-wrong number, which is the most dangerous kind of bug because nobody double-checks a plausible figure.

Testing approach: Data-consistency testing: seed known deal states, compute the expected report value by hand, and diff against the actual report output, including sync-delay and stale-cache scenarios.

Automation candidate: Yes for the calculation-correctness checks (deterministic, easy to assert), but sync-timing/cache-staleness edge cases are worth occasional manual verification since they depend on infrastructure behaviour that can change independently of the report logic.

Task 2.4 — CRM Comparison: QA Perspective

Opinion: HubSpot is the harder platform to test thoroughly, though for different reasons than Zoho is hard in the areas where Zoho does struggle.

- Configuration depth: HubSpot's workflow engine supports branching logic, multiple enrollment triggers, re-enrollment rules, and cross-object actions (contact, deal, company, ticket) in a single automation — the permutation space for "what could this workflow do under condition X" is larger than Zoho's more linear Workflow Rules/Blueprint model.
- Integration/automation surface area: HubSpot is frequently the hub connecting marketing, sales, and service data plus a large marketplace of third-party apps; each integration is a potential point where a property update or association change happens outside the CRM's own UI, which is harder to observe and test than a self-contained Zoho automation.
- Observability: this cuts the other way. Zoho's automation and duplicate-handling failures are, in my experience, quieter and less discoverable (sparser execution logs, fewer in-product error states) than HubSpot's, which at least surfaces workflow history with per-enrollment error detail — so testing Zoho requires more indirect verification (checking side effects rather than a log).
- Net assessment: HubSpot's complexity comes from breadth (more configuration options and integration touchpoints to cover), while Zoho's difficulty comes from opacity (fewer ways to directly confirm something worked). Breadth is the harder problem to fully cover in a fixed test cycle, because opacity can at least be

worked around with disciplined side-effect assertions, whereas breadth genuinely multiplies the number of scenarios that exist.

Part 3 — Test Planning & Organisation

Task 3.1 — Test Plan Outline: "Approval Before Closed Won"

Scope

In scope: the approval gate on the Deal Stage transition to Closed Won, approver notification and escalation, self-approval prevention, re-pointed downstream automations/notifications, forecast/report accuracy post-approval, and bulk import/API enforcement of the same gate.

Out of scope

Out of scope: approval rules on any stage other than Closed Won, mobile-app-specific UI for the approval action (unless separately confirmed in scope by product), and any future multi-level (sequential) approval chains — this release is assumed single-approver.

Assumptions & dependencies

- A designated Approver field/role exists on the Deal or Account and is populated before this feature goes live.
- Existing Workflow Rules tied to Stage = Closed Won have been inventoried by the product/dev team so QA can verify each was re-pointed, rather than discovering them ad hoc.
- A staging environment with representative production-like data (existing open, won, and lost deals) is available for regression testing.

Top 3 risk areas

1. Silent bypass via bulk import/API — the approval gate must be enforced at the data layer, not only the UI.
2. Self-approval — approver and owner being the same person must be explicitly blocked.
3. Downstream automation timing — any process still keyed off raw Stage change instead of Approval Status will fire early.

Entry criteria

- Feature deployed to staging with approver roles configured and at least one end-to-end happy-path approval demoed by dev.
- Inventory of pre-existing Stage-based automations provided by the dev team.

Exit criteria

- All priority-1 test areas (approval gate, self-approval block, bulk-import enforcement) pass with no open Critical/High defects.
- Forecast and quota reports verified to reflect only approved Closed Won deals in at least one full regression pass.
- Any known Medium/Low issues are logged and explicitly accepted by product owner before release.

Environment & data setup needed

Staging instance with: 3+ test users in owner/approver/admin roles (including one user assigned as both owner and approver to test the self-approval edge case), a mix of pre-existing open/won/lost deals to validate legacy-data behaviour, and API/import access enabled for the bypass-testing scenarios.

Task 3.2 — Test Suite Structure: Deal/Opportunity Module

Suite organisation:

- Smoke — Create Deal · move through pipeline stages · close Won · close Lost · reopen a closed deal.
Automation candidate: yes — fast, deterministic, run on every build.
- Regression — Field validation per stage · duplicate/merge behaviour · owner reassignment · activity/email association · automation triggers on stage change · report accuracy after edits. Automation candidate: yes for the majority — stable, well-defined expected outcomes.
- Edge cases — Deal with no owner · Closed Won with empty required fields via import · reopening a Closed Lost deal and checking close-date history · concurrent edits by two users · bulk import overwriting stage.
Automation candidate: partial — logic itself can be scripted, but the "is this visible/surfaced correctly to a user" check often needs manual eyes.
- Integration — Webhook/API payload correctness on stage change · third-party sync (e.g. billing) reflecting Closed Won accurately · email-service BCC logging tied to Deal. Automation candidate: yes for payload/data assertions — manual spot-checks for actual third-party system behaviour.

Example test case (Given/When/Then)

Field	Detail
ID	DEAL-EDGE-014
Title	Deal with empty required field cannot be saved as Closed Won via import
Type	Edge case — manual (data-layer validation, not yet automation-covered)
Given	A Deal exists in an open stage with the Amount field blank
When	A bulk import updates that Deal's Stage to Closed Won without supplying a value for Amount
Then	The import should either reject the row with a validation error, or the Deal should remain in its prior stage with a logged import error — it must not be saved as Closed Won with Amount empty

Task 3.3 — Prioritisation Under Time Pressure (4 Hours Before Release)

Time	Focus
Hour 1	Smoke test the core happy path: create Deal → stage progression → Closed Won with approval → Closed Lost → reopen. Confirms nothing fundamental is broken.
Hour 2	Highest-risk edge cases only: self-approval block, bulk-import bypass of the approval gate, and the one existing automation known to be re-pointed to Approval Status. These are the scenarios most likely to cause silent, hard-to-detect production incidents.
Hour 3	Report/forecast spot-check: confirm Closed Won revenue reports only reflect approved deals, and check one legacy deal (pre-feature) behaves sensibly.

Hour 4	Regression pass on anything the dev team flagged as touched by this change, plus writing up findings and open risks for the release call.
--------	---

Explicitly deferred

- Full regression of every existing Workflow Rule in the org — only the one confirmed to interact with Closed Won is checked.
- Approver-on-leave escalation timing (multi-day wait) — not practically testable within 4 hours, flagged as untested.
- Cross-browser/mobile UI verification of the approval action.

Risks communicated as untested

I would tell the team explicitly: escalation/delegation behaviour for an absent approver is unverified, the full inventory of legacy automations has not been regression-tested end-to-end, and mobile UI for the approval step has not been checked. If any of these are release-blocking in the team's judgment, that's a call for the release owner, not something I'd quietly skip without flagging.

Task 3.4 — Defect Communication

Most impactful issue found: a Deal can be saved as Closed Won with a required field (Amount) empty when written via bulk import, bypassing the validation enforced in the interactive UI.

Field	Detail
Title	Bulk import allows Deal to be marked Closed Won with Amount field empty, bypassing UI-enforced required-field validation
Severity	Critical — directly corrupts revenue-reporting data with no user-facing error, and is not self-evident without deliberately auditing import history
Priority	P1 — must fix before the Approval-Gate release ships, since the same bypass pattern would let the new approval requirement itself be skipped via import
Environment	Staging, Deals module, bulk import (CSV) feature; reproduced on the current release candidate build
Steps to reproduce	1) Prepare a CSV import file for the Deals module with Stage = Closed Won and the Amount column left blank for one row. 2) Run the import with standard field mapping (Amount mapped to Amount). 3) Open the imported Deal record.
Actual result	The Deal is saved with Stage = Closed Won and Amount empty/null; the import summary reports the row as successful with no warning.
Expected result	The import should either reject that row with a clear validation error ("Amount is required to close a deal as Won") or refuse to set Stage = Closed Won while leaving Amount blank, matching the validation already enforced when closing a deal through the UI.
Business impact	Revenue and win-rate reports can silently include Won deals with no dollar value, understating recognized revenue or, if such rows are excluded from aggregation entirely, inflating win-rate percentages by hiding incomplete records among genuine wins. Because imports are a common path for sales-ops teams migrating or bulk-updating data, this is a realistic, repeatable data-integrity gap that directly undermines the trustworthiness of pipeline reporting.